

5_Python_Project_Extracting_and_Visualising Stock Data

January 30, 2024

Extracting and Visualising Stock Data

Description

Extracting essential data from a dataset and displaying it is a necessary part of data science; therefore individuals can make correct decisions based on the data. In this assignment, you will extract some stock data, you will then display this data in a graph.

Table of Contents

- Define a Function that Makes a Graph
- Question 1: Use yfinance to Extract Stock Data
- Question 2: Use Webscraping to Extract Tesla Revenue Data
- Question 3: Use yfinance to Extract Stock Data
- Question 4: Use Webscraping to Extract GME Revenue Data
- Question 5: Plot Tesla Stock Graph
- Question 6: Plot GameStop Stock Graph

Estimated Time Needed: 30 min

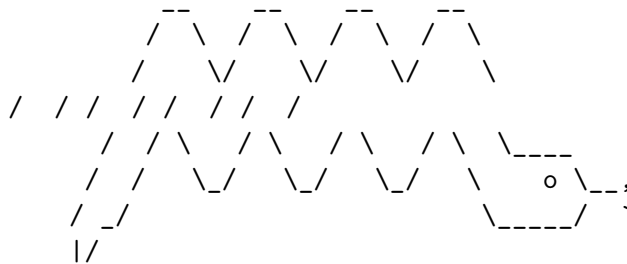
Note:- If you are working in IBM Cloud Watson Studio, please replace the command for installing nbformat from `!pip install nbformat==4.2.0` to simply `!pip install nbformat`

```
[1]: !pip install yfinance==0.2.4
!mamba install bs4==4.10.0 -y
!pip install nbformat==5.2.0
!mamba install html5lib==1.1 -y
!pip install lxml==4.9.1
```

```
Requirement already satisfied: yfinance==0.2.4 in
/home/jupyterlab/conda/envs/python/lib/python3.7/site-packages (0.2.4)
Requirement already satisfied: pandas>=1.3.0 in
/home/jupyterlab/conda/envs/python/lib/python3.7/site-packages (from
yfinance==0.2.4) (1.3.5)
Requirement already satisfied: numpy>=1.16.5 in
/home/jupyterlab/conda/envs/python/lib/python3.7/site-packages (from
yfinance==0.2.4) (1.21.6)
Requirement already satisfied: requests>=2.26 in
/home/jupyterlab/conda/envs/python/lib/python3.7/site-packages (from
```

yfinance==0.2.4) (2.29.0)
Requirement already satisfied: multitasking>=0.0.7 in
/home/jupyterlab/conda/envs/python/lib/python3.7/site-packages (from
yfinance==0.2.4) (0.0.11)
Requirement already satisfied: lxml>=4.9.1 in
/home/jupyterlab/conda/envs/python/lib/python3.7/site-packages (from
yfinance==0.2.4) (4.9.1)
Requirement already satisfied: appdirs>=1.4.4 in
/home/jupyterlab/conda/envs/python/lib/python3.7/site-packages (from
yfinance==0.2.4) (1.4.4)
Requirement already satisfied: pytz>=2022.5 in
/home/jupyterlab/conda/envs/python/lib/python3.7/site-packages (from
yfinance==0.2.4) (2023.3)
Requirement already satisfied: frozendict>=2.3.4 in
/home/jupyterlab/conda/envs/python/lib/python3.7/site-packages (from
yfinance==0.2.4) (2.3.8)
Requirement already satisfied: cryptography>=3.3.2 in
/home/jupyterlab/conda/envs/python/lib/python3.7/site-packages (from
yfinance==0.2.4) (38.0.2)
Requirement already satisfied: beautifulsoup4>=4.11.1 in
/home/jupyterlab/conda/envs/python/lib/python3.7/site-packages (from
yfinance==0.2.4) (4.12.2)
Requirement already satisfied: html5lib>=1.1 in
/home/jupyterlab/conda/envs/python/lib/python3.7/site-packages (from
yfinance==0.2.4) (1.1)
Requirement already satisfied: soupsieve>1.2 in
/home/jupyterlab/conda/envs/python/lib/python3.7/site-packages (from
beautifulsoup4>=4.11.1->yfinance==0.2.4) (2.3.2.post1)
Requirement already satisfied: cffi>=1.12 in
/home/jupyterlab/conda/envs/python/lib/python3.7/site-packages (from
cryptography>=3.3.2->yfinance==0.2.4) (1.15.1)
Requirement already satisfied: six>=1.9 in
/home/jupyterlab/conda/envs/python/lib/python3.7/site-packages (from
html5lib>=1.1->yfinance==0.2.4) (1.16.0)
Requirement already satisfied: webencodings in
/home/jupyterlab/conda/envs/python/lib/python3.7/site-packages (from
html5lib>=1.1->yfinance==0.2.4) (0.5.1)
Requirement already satisfied: python-dateutil>=2.7.3 in
/home/jupyterlab/conda/envs/python/lib/python3.7/site-packages (from
pandas>=1.3.0->yfinance==0.2.4) (2.8.2)
Requirement already satisfied: charset-normalizer<4,>=2 in
/home/jupyterlab/conda/envs/python/lib/python3.7/site-packages (from
requests>=2.26->yfinance==0.2.4) (3.1.0)
Requirement already satisfied: idna<4,>=2.5 in
/home/jupyterlab/conda/envs/python/lib/python3.7/site-packages (from
requests>=2.26->yfinance==0.2.4) (3.4)
Requirement already satisfied: urllib3<1.27,>=1.21.1 in
/home/jupyterlab/conda/envs/python/lib/python3.7/site-packages (from

```
requests>=2.26->yfinance==0.2.4) (1.26.15)
Requirement already satisfied: certifi>=2017.4.17 in
/home/jupyterlab/conda/envs/python/lib/python3.7/site-packages (from
requests>=2.26->yfinance==0.2.4) (2023.5.7)
Requirement already satisfied: pycparser in
/home/jupyterlab/conda/envs/python/lib/python3.7/site-packages (from
cffi>=1.12->cryptography>=3.3.2->yfinance==0.2.4) (2.21)
```



mamba (1.4.2) supported by @QuantStack

GitHub: <https://github.com/mamba-org/mamba>
 Twitter: <https://twitter.com/QuantStack>

Looking for: ['bs4==4.10.0']

```
[+] 0.0s
[+] 0.1s
pkgs/main/linux-64 0.0 B
/ ???.?MB @ ???.?MB/s 0.1s
pkgs/main/noarch 0.0 B / ???.?MB
@ ???.?MB/s 0.1s
pkgs/r/linux-64 0.0 B / ???.?MB
@ ???.?MB/s 0.1s
pkgs/r/noarch 0.0 B
/ ???.?MB @ ???.?MB/s
0.1spkgs/main/noarch
No change
pkgs/r/noarch No change
pkgs/main/linux-64 No change
pkgs/r/linux-64 No change
```

Pinned packages:

- python 3.7.*

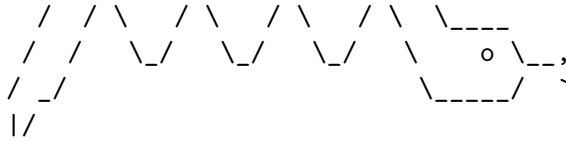
Transaction

Prefix: /home/jupyterlab/conda/envs/python

All requested packages already installed

Requirement already satisfied: nbformat==5.2.0 in
/home/jupyterlab/conda/envs/python/lib/python3.7/site-packages (5.2.0)
Requirement already satisfied: traitlets>=4.1 in
/home/jupyterlab/conda/envs/python/lib/python3.7/site-packages (from
nbformat==5.2.0) (5.9.0)
Requirement already satisfied: jsonschema!=2.5.0,>=2.4 in
/home/jupyterlab/conda/envs/python/lib/python3.7/site-packages (from
nbformat==5.2.0) (4.17.3)
Requirement already satisfied: jupyter-core in
/home/jupyterlab/conda/envs/python/lib/python3.7/site-packages (from
nbformat==5.2.0) (4.12.0)
Requirement already satisfied: attrs>=17.4.0 in
/home/jupyterlab/conda/envs/python/lib/python3.7/site-packages (from
jsonschema!=2.5.0,>=2.4->nbformat==5.2.0) (23.1.0)
Requirement already satisfied: importlib-metadata in
/home/jupyterlab/conda/envs/python/lib/python3.7/site-packages (from
jsonschema!=2.5.0,>=2.4->nbformat==5.2.0) (4.11.4)
Requirement already satisfied: importlib-resources>=1.4.0 in
/home/jupyterlab/conda/envs/python/lib/python3.7/site-packages (from
jsonschema!=2.5.0,>=2.4->nbformat==5.2.0) (5.12.0)
Requirement already satisfied: pkgutil-resolve-name>=1.3.10 in
/home/jupyterlab/conda/envs/python/lib/python3.7/site-packages (from
jsonschema!=2.5.0,>=2.4->nbformat==5.2.0) (1.3.10)
Requirement already satisfied: pyparsing!=0.17.0,!0.17.1,!0.17.2,>=0.14.0 in
/home/jupyterlab/conda/envs/python/lib/python3.7/site-packages (from
jsonschema!=2.5.0,>=2.4->nbformat==5.2.0) (0.19.3)
Requirement already satisfied: typing-extensions in
/home/jupyterlab/conda/envs/python/lib/python3.7/site-packages (from
jsonschema!=2.5.0,>=2.4->nbformat==5.2.0) (4.5.0)
Requirement already satisfied: zipp>=3.1.0 in
/home/jupyterlab/conda/envs/python/lib/python3.7/site-packages (from importlib-
resources>=1.4.0->jsonschema!=2.5.0,>=2.4->nbformat==5.2.0) (3.15.0)

```
      --  --  --  --
     /  \ /  \ /  \ /  \
    /    \    \    \    \
   /      \      \      \
  /        \        \        \
 /          \          \          \
/            \            \            \
```



mamba (1.4.2) supported by @QuantStack

GitHub: <https://github.com/mamba-org/mamba>

Twitter: <https://twitter.com/QuantStack>

Looking for: ['html5lib==1.1']

pkgs/main/linux-64	Using cache
pkgs/main/noarch	Using cache
pkgs/r/linux-64	Using cache
pkgs/r/noarch	Using cache

Pinned packages:

- python 3.7.*

Transaction

Prefix: /home/jupyterlab/conda/envs/python

All requested packages already installed

Requirement already satisfied: lxml==4.9.1 in
/home/jupyterlab/conda/envs/python/lib/python3.7/site-packages (4.9.1)

```
[2]: import yfinance as yf
import pandas as pd
import requests
from bs4 import BeautifulSoup
import plotly.graph_objects as go
from plotly.subplots import make_subplots
```

0.1 Define Graphing Function

In this section, we define the function `make_graph`. You don't have to know how the function works, you should only care about the inputs. It takes a dataframe with stock data (dataframe must contain Date and Close columns), a dataframe with revenue data (dataframe must contain Date and Revenue columns), and the name of the stock.

```
[3]: def make_graph(stock_data, revenue_data, stock):
    fig = make_subplots(rows = 2, cols = 1, shared_xaxes = True, subplot_titles_
    ↪ ("Historical Share Price", "Historical Revenue"), vertical_spacing = .3)
    stock_data_specific = stock_data[stock_data.Date <= '2021-06-14']
    revenue_data_specific = revenue_data[revenue_data.Date <= '2021-04-30']
    fig.add_trace(go.Scatter(x = pd.to_datetime(stock_data_specific.Date,
    ↪ infer_datetime_format = True), y = stock_data_specific.Close.
    ↪ astype("float"), name = "Share Price"), row = 1, col = 1)
    fig.add_trace(go.Scatter(x = pd.to_datetime(revenue_data_specific.Date,
    ↪ infer_datetime_format = True), y = revenue_data_specific.Revenue.
    ↪ astype("float"), name = "Revenue"), row = 2, col = 1)
    fig.update_xaxes(title_text = "Date", row = 1, col = 1)
    fig.update_xaxes(title_text = "Date", row = 2, col = 1)
    fig.update_yaxes(title_text = "Price ($US)", row = 1, col = 1)
    fig.update_yaxes(title_text = "Revenue ($US Millions)", row = 2, col = 1)
    fig.update_layout(showlegend = False,
    height = 900,
    title = stock,
    xaxis_rangeslider_visible = True)
    fig.show()
```

0.2 Question 1: Use yfinance to Extract Stock Data

Using the Ticker function enter the ticker symbol of the stock we want to extract data on to create a ticker object. The stock is Tesla and its ticker symbol is TSLA.

```
[4]: tesla = yf.Ticker('TSLA')
```

Using the ticker object and the function `history` extract stock information and save it in a dataframe named `tesla_data`. Set the `period` parameter to `max` so we get information for the maximum amount of time.

```
[5]: tesla_data = tesla.history(period = 'max')
```

```
[6]: tesla_data.head()
```

```
[6]:
```

	Open	High	Low	Close	Volume \
Date					
2010-06-29 00:00:00-04:00	1.266667	1.666667	1.169333	1.592667	281494500
2010-06-30 00:00:00-04:00	1.719333	2.028000	1.553333	1.588667	257806500
2010-07-01 00:00:00-04:00	1.666667	1.728000	1.351333	1.464000	123282000

2010-07-02 00:00:00-04:00	1.533333	1.540000	1.247333	1.280000	77097000
2010-07-06 00:00:00-04:00	1.333333	1.333333	1.055333	1.074000	103003500

	Dividends	Stock Splits
Date		
2010-06-29 00:00:00-04:00	0.0	0.0
2010-06-30 00:00:00-04:00	0.0	0.0
2010-07-01 00:00:00-04:00	0.0	0.0
2010-07-02 00:00:00-04:00	0.0	0.0
2010-07-06 00:00:00-04:00	0.0	0.0

Reset the index using the `reset_index(inplace=True)` function on the `tesla_data` DataFrame and display the first five rows of the `tesla_data` dataframe using the `head` function. Take a screenshot of the results and code from the beginning of Question 1 to the results below.

```
[7]: tesla_data.reset_index(inplace = True)
```

```
[8]: tesla_data.head()
```

```
[8]:
```

	Date	Open	High	Low	Close \
0	2010-06-29 00:00:00-04:00	1.266667	1.666667	1.169333	1.592667
1	2010-06-30 00:00:00-04:00	1.719333	2.028000	1.553333	1.588667
2	2010-07-01 00:00:00-04:00	1.666667	1.728000	1.351333	1.464000
3	2010-07-02 00:00:00-04:00	1.533333	1.540000	1.247333	1.280000
4	2010-07-06 00:00:00-04:00	1.333333	1.333333	1.055333	1.074000

	Volume	Dividends	Stock Splits
0	281494500	0.0	0.0
1	257806500	0.0	0.0
2	123282000	0.0	0.0
3	77097000	0.0	0.0
4	103003500	0.0	0.0

0.3 Question 2: Use Webscraping to Extract Tesla Revenue Data

Use the `requests` library to download the webpage <https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/IBMDeveloperSkillsNetwork-PY0220EN-SkillsNetwork/labs/project/revenue.htm> Save the text of the response as a variable named `html_data`.

```
[9]: url = "https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/
↳IBMDeveloperSkillsNetwork-PY0220EN-SkillsNetwork/labs/project/revenue.htm"
```

```
[10]: html_data = requests.get(url).text
```

Parse the html data using `beautiful_soup`.

```
[11]: soup = BeautifulSoup(html_data, 'html5lib')
```

Using BeautifulSoup or the `read_html` function extract the table with Tesla Revenue and store it into a dataframe named `tesla_revenue`. The dataframe should have columns `Date` and `Revenue`.

[Click here](#) if you need help locating the table

Below is the code to isolate the table, you will now need to loop through the rows and columns

```
soup.find_all("tbody")[1]
```

If you want to use the `read_html` function the table is located at index 1

```
[12]: tesla_revenue = pd.DataFrame(columns=['Date', 'Revenue'])
```

```
[13]: for row in soup.find_all('tbody')[1].find_all('tr'):
      col = row.find_all('td')
      date = col[0].text
      revenue = col[1].text
      tesla_revenue = tesla_revenue.append({'Date': date, 'Revenue': revenue},
      ↪ ignore_index = True)
```

```
[14]: tesla_revenue.head()
```

```
[14]:
```

	Date	Revenue
0	2022-09-30	\$21,454
1	2022-06-30	\$16,934
2	2022-03-31	\$18,756
3	2021-12-31	\$17,719
4	2021-09-30	\$13,757

Execute the following line to remove the comma and dollar sign from the `Revenue` column.

```
[15]: tesla_revenue['Revenue'] = tesla_revenue['Revenue'].str.replace(',', '\\$', '',
      ↪ regex = True)
```

```
[16]: tesla_revenue.head()
```

```
[16]:
```

	Date	Revenue
0	2022-09-30	21454
1	2022-06-30	16934
2	2022-03-31	18756
3	2021-12-31	17719
4	2021-09-30	13757

Execute the following lines to remove an null or empty strings in the `Revenue` column.

```
[17]: tesla_revenue.dropna(inplace = True)
      tesla_revenue = tesla_revenue[tesla_revenue['Revenue'] != '']
```


Display the last 5 row of the `tesla_revenue` dataframe using the `tail` function. Take a screenshot of the results.

```
[18]: tesla_revenue.tail()
```

```
[18]:
```

	Date	Revenue
48	2010-09-30	31
49	2010-06-30	28
50	2010-03-31	21
52	2009-09-30	46
53	2009-06-30	27

0.4 Question 3: Use yfinance to Extract Stock Data

Using the `Ticker` function enter the ticker symbol of the stock we want to extract data on to create a ticker object. The stock is GameStop and its ticker symbol is `GME`.

```
[19]: gme = yf.Ticker('GME')
```

Using the ticker object and the function `history` extract stock information and save it in a dataframe named `gme_data`. Set the `period` parameter to `max` so we get information for the maximum amount of time.

```
[20]: gme_data = gme.history(period = 'max')
```

Reset the index using the `reset_index(inplace=True)` function on the `gme_data` DataFrame and display the first five rows of the `gme_data` dataframe using the `head` function. Take a screenshot of the results and code from the beginning of Question 3 to the results below.

```
[21]: gme_data.head()
```

```
[21]:
```

		Open	High	Low	Close	Volume \
Date						
2002-02-13 00:00:00-05:00		1.620128	1.693350	1.603296	1.691667	76216000
2002-02-14 00:00:00-05:00		1.712707	1.716074	1.670626	1.683250	11021600
2002-02-15 00:00:00-05:00		1.683250	1.687458	1.658002	1.674834	8389600
2002-02-19 00:00:00-05:00		1.666418	1.666418	1.578047	1.607504	7410400
2002-02-20 00:00:00-05:00		1.615921	1.662210	1.603296	1.662210	6892800

	Dividends	Stock Splits
Date		
2002-02-13 00:00:00-05:00	0.0	0.0
2002-02-14 00:00:00-05:00	0.0	0.0
2002-02-15 00:00:00-05:00	0.0	0.0
2002-02-19 00:00:00-05:00	0.0	0.0
2002-02-20 00:00:00-05:00	0.0	0.0

```
[22]: gme_data.reset_index(inplace = True)
```

```
[23]: gme_data.head()
```

```
[23]:
```

	Date	Open	High	Low	Close	Volume	\
0	2002-02-13 00:00:00-05:00	1.620128	1.693350	1.603296	1.691667	76216000	
1	2002-02-14 00:00:00-05:00	1.712707	1.716074	1.670626	1.683250	11021600	
2	2002-02-15 00:00:00-05:00	1.683250	1.687458	1.658002	1.674834	8389600	
3	2002-02-19 00:00:00-05:00	1.666418	1.666418	1.578047	1.607504	7410400	
4	2002-02-20 00:00:00-05:00	1.615921	1.662210	1.603296	1.662210	6892800	

	Dividends	Stock Splits
0	0.0	0.0
1	0.0	0.0
2	0.0	0.0
3	0.0	0.0
4	0.0	0.0

0.5 Question 4: Use Webscraping to Extract GME Revenue Data

Use the `requests` library to download the webpage <https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/IBMDeveloperSkillsNetwork-PY0220EN-SkillsNetwork/labs/project/stock.html>. Save the text of the response as a variable named `html_data`.

```
[24]: url = "https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/  
↪IBMDeveloperSkillsNetwork-PY0220EN-SkillsNetwork/labs/project/stock.html"
```

```
[25]: html_data = requests.get(url).text
```

Parse the html data using `beautiful_soup`.

```
[26]: soup = BeautifulSoup(html_data, 'html5lib')
```

Using `BeautifulSoup` or the `read_html` function extract the table with **GameStop Revenue** and store it into a dataframe named `gme_revenue`. The dataframe should have columns `Date` and `Revenue`. Make sure the comma and dollar sign is removed from the `Revenue` column using a method similar to what you did in Question 2.

[Click here](#) if you need help locating the table

Below is the code to isolate the table, you will now need to loop through the rows and columns

```
soup.find_all("tbody")[1]
```

If you want to use the `read_html` function the table is located at index 1

```
[27]: gme_revenue = pd.DataFrame(columns=['Date', 'Revenue'])
```

```
[28]: for row in soup.find_all('tbody')[1].find_all('tr'):
        col = row.find_all('td')
        date = col[0].text
        revenue = col[1].text
        gme_revenue = gme_revenue.append({'Date': date, 'Revenue': revenue},
        ignore_index = True)
```

```
[29]: gme_revenue.head()
```

```
[29]:      Date Revenue
0  2020-04-30  $1,021
1  2020-01-31  $2,194
2  2019-10-31  $1,439
3  2019-07-31  $1,286
4  2019-04-30  $1,548
```

```
[30]: gme_revenue['Revenue'] = gme_revenue['Revenue'].str.replace(',|\$','', regex =
        True)
```

```
[31]: gme_revenue.dropna(inplace = True)
gme_revenue = gme_revenue[gme_revenue['Revenue'] != '']
```

Display the last five rows of the `gme_revenue` dataframe using the `tail` function. Take a screenshot of the results.

```
[32]: gme_revenue.tail()
```

```
[32]:      Date Revenue
57  2006-01-31    1667
58  2005-10-31     534
59  2005-07-31     416
60  2005-04-30     475
61  2005-01-31     709
```

0.6 Question 5: Plot Tesla Stock Graph

Use the `make_graph` function to graph the Tesla Stock Data, also provide a title for the graph. The structure to call the `make_graph` function is `make_graph(tesla_data, tesla_revenue, 'Tesla')`. Note the graph will only show data up to June 2021.

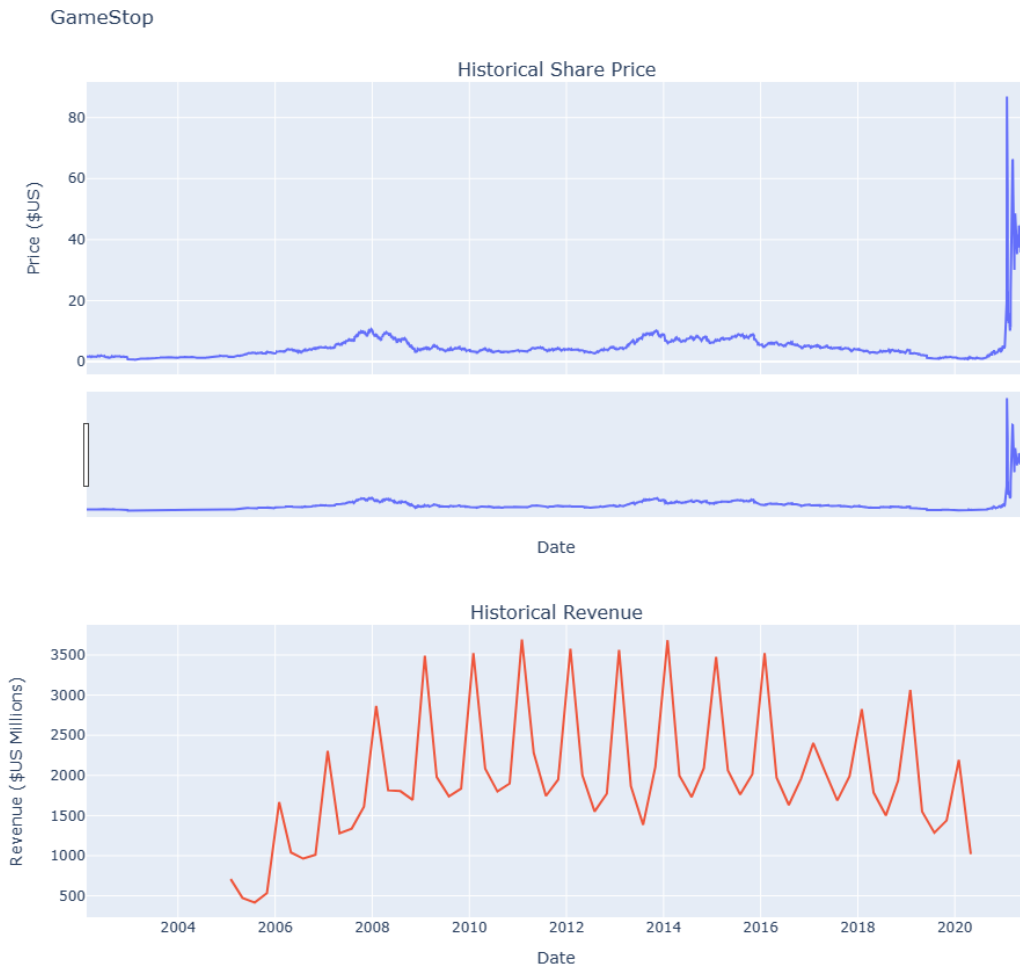
```
[33]: make_graph(tesla_data, tesla_revenue, 'Tesla')
```



0.7 Question 6: Plot GameStop Stock Graph

Use the `make_graph` function to graph the GameStop Stock Data, also provide a title for the graph. The structure to call the `make_graph` function is `make_graph(gme_data, gme_revenue, 'GameStop')`. Note the graph will only show data upto June 2021.

```
[34]: make_graph(gme_data, gme_revenue, 'GameStop')
```



About the Authors:

Joseph Santarcangelo has a PhD in Electrical Engineering, his research focused on using machine learning, signal processing, and computer vision to determine how videos impact human cognition. Joseph has been working for IBM since he completed his PhD.

Azim Hirjani

0.8 Change Log

Date (YYYY-MM-DD)	Version	Changed By	Change Description
2022-02-28	1.2	Lakshmi Holla	Changed the URL of GameStop
2020-11-10	1.1	Malika Singla	Deleted the Optional part
2020-08-27	1.0	Malika Singla	Added lab to GitLab

##

© IBM Corporation 2020. All rights reserved.